

# GoHighLevel Interface Specification

API, Data Exchange and Integration Architecture

Investment Property Marketplace — EspoCRM Replacement Programme

|                         |                                                          |
|-------------------------|----------------------------------------------------------|
| Document type           | Integration interface specification                      |
| Target system           | GoHighLevel (HighLevel) CRM — Public API v2              |
| API base URL            | <code>https://services.leadconnectorhq.com</code>        |
| Required version header | Version: 2021-07-28                                      |
| Companion system        | PostgreSQL 16 (migration staging and system of record)   |
| Deployment scope        | Local development and testing only; no internet exposure |
| Status                  | Draft for review                                         |

*Every endpoint path, HTTP method, OAuth scope, request header, object field and webhook event name in this document was read directly from GoHighLevel's official OpenAPI source specifications at [github.com/GoHighLevel/highlevel-api-docs](https://github.com/GoHighLevel/highlevel-api-docs). Nothing here is reproduced from marketing material or recollection. Where a behaviour could not be verified against those specifications it is explicitly flagged as *unverified* rather than *asserted*.*

# 1. Purpose and Scope

This document defines the programmatic interface between GoHighLevel and the systems around it for a private investment property marketplace: properties as the core entity, linked to investors, agents and lenders, with a gated portal behind a platform fee agreement.

It is written to answer three questions precisely: what GHL's API can and cannot do, how transactional and relational data moves in each direction, and which parts of the intended design are constrained by the platform.

## 1.1 In scope

| Area           | Covered                                                            |
|----------------|--------------------------------------------------------------------|
| Authentication | OAuth 2.0 authorization code flow and Private Integration Tokens   |
| Transport      | Base URL, required headers, content types, pagination, rate limits |
| Data model     | Custom objects, associations, contacts, opportunities              |
| Transactional  | Invoices, payments, orders, subscriptions, transactions            |
| Events         | Webhook catalogue, signature verification, replay defence          |
| Migration      | EspoCRM to GHL relational load ordering and staging schema         |
| Constraints    | Platform limits that materially affect the intended architecture   |

## 1.2 Out of scope

| Area                                        | Reason                                           |
|---------------------------------------------|--------------------------------------------------|
| Internet exposure, tunnels, TLS termination | Explicitly deferred; local-only in this phase    |
| Second brand rollout                        | Separate future phase, not part of this project  |
| GHL v1 (API-key) endpoints                  | Legacy; absent from the current v2 specification |
| Marketplace app listing and billing         | Not required for a single-tenant integration     |

# 2. Domain Model Mapping

GHL is contact-centric by design. An entity-centric marketplace, where the primary object is a property rather than a person, is achievable through Custom Objects, but the mapping is not free of friction and should be understood before build.

| Domain entity               | GHL representation             | API surface                  |
|-----------------------------|--------------------------------|------------------------------|
| Property / listing          | Custom Object record           | /objects/{schemaKey}/records |
| Investor                    | Contact                        | /contacts/upsert             |
| Agent                       | User, or Contact with role tag | /users/, /contacts/          |
| Lender                      | Contact with role tag          | /contacts/upsert             |
| Deal / transaction pipeline | Opportunity in a Pipeline      | /opportunities/upsert        |

| Domain entity            | GHL representation                  | API surface            |
|--------------------------|-------------------------------------|------------------------|
| Property → Investor link | Association / Relation              | /associations/reasons  |
| Platform fee agreement   | Invoice plus e-signature document   | /invoices/             |
| Fee payment              | Transaction (read-only)             | /payments/transactions |
| Saved property           | Association or contact custom field | /associations/reasons  |

## 2.1 Custom Objects and Associations

| Method         | Path                                | Purpose                                          |
|----------------|-------------------------------------|--------------------------------------------------|
| GET/POST       | /objects/                           | List or create object schemas                    |
| GET/PUT        | /objects/{key}                      | Read or update a schema                          |
| POST           | /objects/{schemaKey}/records        | Create a record                                  |
| GET/PUT/DELETE | /objects/{schemaKey}/records/{id}   | Read, update, delete a record                    |
| POST           | /objects/{schemaKey}/records/search | Search records — the marketplace query primitive |
| POST/GET       | /associations/                      | Define association types between objects         |
| POST           | /associations/reasons               | Create a relation between two records            |
| GET            | /associations/reasons/{recordId}    | List relations for a record                      |
| DELETE         | /associations/reasons/{relationId}  | Remove a relation                                |

Associations are the mechanism that preserves the relational integrity of an EspoCRM migration. They are created as a discrete second pass after both sides of the relationship exist, which dictates the load ordering in section 8.

## 2.2 Opportunities (deal pipelines)

| Method         | Path                          | Purpose                                         |
|----------------|-------------------------------|-------------------------------------------------|
| POST           | /opportunities/               | Create opportunity                              |
| POST           | /opportunities/upsert         | Idempotent create-or-update; preferred for sync |
| GET/PUT/DELETE | /opportunities/{id}           | Read, update, delete                            |
| PUT            | /opportunities/{id}/status    | Advance or close a deal                         |
| GET/POST       | /opportunities/search         | Query across pipelines                          |
| GET            | /opportunities/pipelines      | List pipelines and stages                       |
| GET            | /opportunities/lost-reason    | Loss reason taxonomy                            |
| POST/DELETE    | /opportunities/{id}/followers | Assign watchers, e.g. agent or lender           |

### 3. Transport and Versioning

| Property                    | Value                                             |
|-----------------------------|---------------------------------------------------|
| Base URL                    | https://services.leadconnectorhq.com              |
| Protocol                    | REST over HTTPS, JSON request and response bodies |
| Version header              | Version: 2021-07-28                               |
| Authorization header        | Authorization: Bearer <token>                     |
| Token format                | JWT bearer                                        |
| Token endpoint content type | application/x-www-form-urlencoded                 |
| Pagination                  | limit and offset query parameters                 |

**The Version header is not advisory. In the OpenAPI specification it is a required header parameter with a single-value enum of 2021-07-28; requests omitting it are rejected. Set it once in a shared HTTP client rather than at each call site.**

```
GET /payments/transactions?altId=<locationId>&altType=location&limit=100&offset=0
Host: services.leadconnectorhq.com
Authorization: Bearer <access_token>
Version: 2021-07-28
Accept: application/json
```

### 4. Authentication

Two credential types are accepted. Both are presented as a bearer token; they differ in acquisition and maintenance.

#### 4.1 Option A — Private Integration Token (recommended for this phase)

A static, long-lived token issued per sub-account from the GHL interface. The OpenAPI security schemes accept it wherever a sub-account OAuth token is accepted: *“Use the Access Token generated with user type as Sub-Account (OR) Private Integration Token of Sub-Account.”*

- No authorization redirect and no callback URL, therefore no inbound HTTP requirement.
- No refresh loop and no token rotation state to persist.
- Scoped to a single sub-account, which matches a single-brand deployment.
- Directly compatible with local-only development, since nothing must be reachable from the internet.

**This is the correct choice for the current phase. Moving to OAuth later changes credential acquisition only; every downstream request is unchanged.**

#### 4.2 Option B — OAuth 2.0 Authorization Code

Required only if the integration is distributed as a Marketplace app serving multiple agencies or sub-accounts.

| Step         | Endpoint                                                                                                       |
|--------------|----------------------------------------------------------------------------------------------------------------|
| 1. Authorize | https://marketplace.gohighlevel.com/v2/oauth/chooselocation<br>Alternate host: marketplace.leadconnectorhq.com |

| Step                  | Endpoint                                        |
|-----------------------|-------------------------------------------------|
| 2. Exchange code      | POST /oauth/token — form-encoded body           |
| 3. Refresh            | POST /oauth/token with grant_type=refresh_token |
| 4. Agency to location | POST /oauth/locationToken                       |
| 5. Discover installs  | GET /oauth/installedLocations                   |

## Token request fields

| Field         | Required    | Notes                                                   |
|---------------|-------------|---------------------------------------------------------|
| client_id     | Yes         | Issued by GHL                                           |
| client_secret | Yes         | Issued by GHL                                           |
| grant_type    | Yes         | authorization_code   refresh_token   client_credentials |
| code          | Conditional | Authorization code exchange                             |
| refresh_token | Conditional | Refresh calls                                           |
| user_type     | Conditional | Company (agency) or Location (sub-account)              |
| redirect_uri  | Conditional | Must match the registered callback                      |

**Access tokens are valid for approximately 24 hours. Refresh tokens rotate: each refresh returns a new access token and a new refresh token, and the previous refresh token is spent. Persist both atomically or the integration will lock itself out.**

## 4.3 Scopes relevant to this build

| Scope                              | Grants                                           |
|------------------------------------|--------------------------------------------------|
| payments/transactions.readonly     | List and read transactions                       |
| payments/orders.readonly           | List and read orders and fulfillments            |
| payments/orders.collectPayment     | Record a payment against an order                |
| payments/subscriptions.readonly    | List and read subscriptions                      |
| payments/custom-provider.write     | Register and configure a custom payment provider |
| invoices.readonly / invoices.write | Read; create, update, send, void, record payment |
| invoices/schedule.write            | Create and manage recurring schedules            |
| invoices/estimate.write            | Manage estimates and convert to invoice          |

## 5. Rate Limits

| Limit | Value                     | Granularity                               |
|-------|---------------------------|-------------------------------------------|
| Burst | 100 requests / 10 seconds | Per app (client), per Location or Company |
| Daily | 200,000 requests / day    | Per app (client), per Location or Company |

  

| Response header                   | Meaning                                |
|-----------------------------------|----------------------------------------|
| X-RateLimit-Max                   | Maximum requests in the burst interval |
| X-RateLimit-Interval-Milliseconds | Burst interval width                   |
| X-RateLimit-Limit-Daily           | Daily ceiling                          |
| X-RateLimit-Daily-Remaining       | Requests remaining today               |

These limits are comfortable for back-office automation and hostile to a browser-driven public marketplace. A listing page that queries GHL once per visitor shares a single 100-request / 10-second budget across *all* concurrent visitors, because the limit is per application per location, not per user. Thirty simultaneous visitors each triggering four calls will exhaust the burst window.

**The mitigation is architectural, not incidental: property data must be cached in a datastore the front end can query freely, with GHL as the source that populates that cache through webhooks and periodic sync. This is the primary technical reason the PostgreSQL layer in section 8 exists.**

## 6. Transactional Endpoints

**The single most important structural fact about this API is asymmetry: transactions and subscriptions are read-only. No endpoint creates a transaction. Money enters GHL only through an invoice, an order payment record, or a registered custom payment provider.**

### 6.1 Payments

| Method   | Path                                      | Scope                           |
|----------|-------------------------------------------|---------------------------------|
| GET      | /payments/transactions                    | payments/transactions.readonly  |
| GET      | /payments/transactions/{transactionId}    | payments/transactions.readonly  |
| GET      | /payments/orders                          | payments/orders.readonly        |
| GET      | /payments/orders/{orderId}                | payments/orders.readonly        |
| POST     | /payments/orders/{orderId}/record-payment | payments/orders.collectPayment  |
| GET/POST | /payments/orders/{orderId}/fulfillments   | payments/orders.*               |
| GET      | /payments/subscriptions                   | payments/subscriptions.readonly |
| GET      | /payments/subscriptions/{subscriptionId}  | payments/subscriptions.readonly |
| POST     | /payments/custom-provider/provider        | payments/custom-provider.write  |

## 6.2 Invoices — the platform fee mechanism

The platform fee agreement is billed here. Estimates and recurring schedules cover advisory and premium service tiers if those are introduced later.

| Method              | Path                                 | Purpose                                   |
|---------------------|--------------------------------------|-------------------------------------------|
| POST / GET          | /invoices/                           | Create invoice; list invoices             |
| GET/PUT/DELETE      | /invoices/{invoiceId}                | Read, update, delete                      |
| POST                | /invoices/{invoiceId}/send           | Deliver invoice to the investor           |
| POST                | /invoices/{invoiceId}/void           | Void invoice                              |
| POST                | /invoices/{invoiceId}/record-payment | Record an offline payment                 |
| POST                | /invoices/text2pay                   | Create and send a pay-by-link invoice     |
| GET                 | /invoices/generate-invoice-number    | Reserve the next invoice number           |
| POST/GET/PUT/DELETE | /invoices/schedule/...               | Recurring schedules, auto-payment, cancel |
| POST/GET/PUT/DELETE | /invoices/estimate/...               | Estimates; convert estimate to invoice    |
| POST/GET/PUT/DELETE | /invoices/template/...               | Invoice templates and fee configuration   |

## 7. The Transaction Object

Returned by GET /payments/transactions/{transactionId} and in list form by GET /payments/transactions.

| Field                                | Notes                                                                 |
|--------------------------------------|-----------------------------------------------------------------------|
| _id                                  | GHL transaction identifier; natural primary key for the landing table |
| altId / altType                      | Scope; altType is normally location                                   |
| contactId / contactSnapshot          | Links the payment to an investor, with denormalised state             |
| amount / currency                    | Settlement amount; store as integer minor units                       |
| amountRefunded                       | Non-zero indicates partial or full refund                             |
| status                               | Settlement state                                                      |
| liveMode / markAsTest                | Separates live money from test traffic; always filter on this         |
| entityType / entityId / entitySource | What produced the transaction (invoice, order, funnel)                |
| invoiceId                            | Set when the transaction settles an invoice; primary join key         |
| subscriptionId                       | Set for recurring charges                                             |

| Field                                       | Notes                                            |
|---------------------------------------------|--------------------------------------------------|
| chargeId / chargeSnapshot / paymentProvider | Processor reference and identity                 |
| receiptId                                   | Receipt reference for investor-facing records    |
| qboSynced / qboResponse                     | QuickBooks sync state already modelled by GHL    |
| createdAt / updatedAt                       | Timestamps; drive incremental polling from these |
| ipAddress, meta, traceId, createdBy         | Provenance, metadata and audit lineage           |

## List endpoint filters

| Parameter                              | Required | Notes                                            |
|----------------------------------------|----------|--------------------------------------------------|
| altId / altType                        | Yes      | Location id; location                            |
| contactId                              | No       | Filter to one investor                           |
| subscriptionId / entityId              | No       | Filter to a recurring agreement or source entity |
| entitySourceType / entitySourceSubType | No       | e.g. funnel, two_step_order_form                 |
| paymentMode                            | No       | live or test                                     |
| startAt / endAt                        | No       | Date range, e.g. 2026-02-01                      |
| search                                 | No       | Free-text match                                  |
| limit / offset                         | No       | Defaults 10 / 0; use 100 for backfill            |



## 8. Webhooks

GHL publishes **58 event types**. Events are the primary mechanism for keeping an external datastore current; polling exists for backfill and reconciliation.

### 8.1 Events relevant to this build

| Event                            | Use                                                       |
|----------------------------------|-----------------------------------------------------------|
| InvoicePaid                      | Platform fee settled; unlock gated portal access          |
| InvoicePartiallyPaid             | Partial settlement; hold the gate and flag for follow-up  |
| InvoiceSent / InvoiceVoid        | Fee agreement delivered; obligation reversed              |
| InvoiceCreate/Update/Delete      | Keep the invoice mirror in step                           |
| OrderCreate / OrderStatusUpdate  | Non-invoice purchases                                     |
| RecordCreate/Update/Delete       | Property custom object changes; refresh the listing cache |
| AssociationCreate/Update/Delete  | Property to investor, agent or lender links change        |
| RelationCreate/Delete            | Individual relation changes                               |
| ContactCreate/Update/Delete      | Investor identity changes                                 |
| ContactTagUpdate                 | Gate state changes, e.g. Fee_Agreement_Signed             |
| Opportunity*                     | Deal stage, status, owner and value changes               |
| InboundMessage / OutboundMessage | Unified conversation timeline                             |

Remaining events cover appointments, tasks, notes, products, prices, object schema changes, email statistics, users, locations and app lifecycle (AppInstall, AppUninstall, PlanChange).

### 8.2 Signature verification — asymmetric, not a shared secret

**This is the detail most integrations get wrong. GHL does not sign webhooks with an HMAC shared secret. It signs with an RSA private key and publishes the corresponding public key. The receiver verifies the x-wh-signature header against the raw request body using that public key.**

| Element            | Detail                                                                |
|--------------------|-----------------------------------------------------------------------|
| Header             | x-wh-signature                                                        |
| Verification input | The raw, unparsed request body. Never re-serialise the JSON first.    |
| Key                | GHL-published RSA public key (PEM, SPKI)                              |
| Payload fields     | timestamp, webhookId, plus event data                                 |
| Replay window      | Reject if timestamp is outside roughly 5 minutes                      |
| Idempotency        | Reject duplicate webhookId values                                     |
| Key rotation       | GHL rotates the key on notice; keep it configurable, never hard-coded |

```
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.asymmetric import padding
from cryptography.exceptions import InvalidSignature
```

```
import base64, json, datetime

PUBLIC_KEY = serialization.load_pem_public_key(GHL_WEBHOOK_PUBLIC_KEY_PEM)

def verify(raw_body: bytes, signature_b64: str) -> bool:
    try:
        PUBLIC_KEY.verify(
            base64.b64decode(signature_b64),
            raw_body,
            padding.PKCS1v15(),
            hashes.SHA256(),
        )
        return True
    except InvalidSignature:
        return False

def handle(raw_body, headers):
    if not verify(raw_body, headers['x-wh-signature']):
        return 401
    evt = json.loads(raw_body)
    if abs((now_utc() - parse(evt['timestamp'])).total_seconds()) > 300:
        return 400
    if already_seen(evt['webhookId']):
        return 200
    enqueue(evt)
    return 200
```

**The exact signature algorithm and padding are not stated in the published documentation. PKCS#1 v1.5 with SHA-256 is the conventional pairing for this key format and is the recommended first implementation, but it must be confirmed against a captured live webhook before the receiver is trusted. This is open item 1 in section 12.**

## 9. Migration and Synchronisation Design

### 9.1 EspoCRM to GHL load ordering

GHL has no transactional import. Relational integrity is preserved by ordering the load and creating associations as a separate pass once both endpoints of each relationship exist. Staging the extract in PostgreSQL first makes the load restartable and auditable, which a direct CSV or script-to-API load is not.

| Pas<br>s | Action                                                                    | Endpoint                          |
|----------|---------------------------------------------------------------------------|-----------------------------------|
| 0        | Extract EspoCRM to PostgreSQL staging tables; assign stable external keys | —                                 |
| 1        | Create custom object schemas (Property, and any others)                   | POST /objects/                    |
| 2        | Define association types between schemas and contacts                     | POST /associations/               |
| 3        | Load investors, agents and lenders as contacts                            | POST /contacts/upsert             |
| 4        | Load property records                                                     | POST /objects/{schemaKey}/records |
| 5        | Create relations linking properties to contacts                           | POST /associations/relations      |
| 6        | Load deals into pipelines                                                 | POST /opportunities/upsert        |
| 7        | Reconcile: re-read counts and spot-check relations against staging        | /records/search                   |

**Record every GHL id returned in pass 3 to 6 back into the staging tables as the load proceeds. Without that id map, pass 5 cannot be built and a failed load cannot be resumed without duplicating records.**

### 9.2 Ongoing synchronisation

| Path           | Mechanism                                  | Cadence   | Purpose                             |
|----------------|--------------------------------------------|-----------|-------------------------------------|
| Primary        | Webhook events                             | Real time | State changes in both directions    |
| Backfill       | /payments/transactions,<br>/records/search | One-off   | Historical load at go-live          |
| Reconciliation | startAt / endAt sweep                      | Nightly   | Catch missed or out-of-order events |
| Repair         | /payments/transactions/{id}                | On demand | Resolve a specific discrepancy      |

Webhook delivery is not exactly-once and not ordered. The nightly reconciliation sweep is not optional: it is what makes the data trustworthy. Compare a trailing window against the landing tables and raise a discrepancy report rather than silently overwriting.

### 9.3 PostgreSQL staging and landing schema

```
-- Identity bridge. Written during migration, read forever after.
CREATE TABLE ghl_id_map (
  entity_type  text NOT NULL,      -- 'property' | 'investor' | 'agent' | 'deal'
  source_id    text NOT NULL,      -- EspoCRM primary key
  ghl_id       text NOT NULL,
  ghl_object   text NOT NULL,      -- 'record' | 'contact' | 'opportunity'
  location_id  text NOT NULL,
```

```

    migrated_at    timestampz NOT NULL DEFAULT now(),
    PRIMARY KEY (entity_type, source_id, location_id),
    UNIQUE (ghl_id, location_id)
);

-- Append-only event log: the dedupe and audit backbone.
CREATE TABLE ghl_webhook_event (
    webhook_id     text PRIMARY KEY,      -- payload webhookId
    event_type     text NOT NULL,
    occurred_at    timestampz NOT NULL,   -- payload timestamp
    received_at    timestampz NOT NULL DEFAULT now(),
    processed_at   timestampz,
    signature_ok   boolean NOT NULL,
    payload        jsonb NOT NULL
);
CREATE INDEX ON ghl_webhook_event (event_type, occurred_at DESC);
CREATE INDEX ON ghl_webhook_event (processed_at) WHERE processed_at IS NULL;

-- Read model for the public marketplace. Avoids per-visitor GHL calls (sec. 5).
CREATE TABLE property_listing (
    ghl_record_id  text PRIMARY KEY,
    location_id    text NOT NULL,
    status         text NOT NULL,        -- active | pending | sold
    public_visible boolean NOT NULL DEFAULT false,
    display_region text,                 -- coarse location shown publicly
    financials     jsonb NOT NULL,       -- non-sensitive analysis metrics
    restricted     jsonb NOT NULL,       -- exact address etc; never sent unauthenticated
    ghl_updated_at timestampz NOT NULL,
    synced_at      timestampz NOT NULL DEFAULT now()
);
CREATE INDEX ON property_listing (status, public_visible);

-- Mirror of GHL transactions; GHL _id is the natural key.
CREATE TABLE ghl_transaction (
    ghl_id         text PRIMARY KEY,
    location_id    text NOT NULL,
    contact_id     text,
    invoice_id     text,
    subscription_id text,
    amount_minor   bigint NOT NULL,     -- integer minor units, never float
    currency       char(3) NOT NULL,
    amount_refunded_minor bigint NOT NULL DEFAULT 0,
    status         text NOT NULL,
    live_mode      boolean NOT NULL,
    payment_provider text,
    entity_type    text,
    entity_id      text,
    ghl_created_at timestampz NOT NULL,
    ghl_updated_at timestampz NOT NULL,
    raw            jsonb NOT NULL
);
CREATE INDEX ON ghl_transaction (invoice_id);
CREATE INDEX ON ghl_transaction (contact_id);

```

**Store money as `bigint` minor units. Never use floating point for currency, and prefer an explicit integer column over `numeric` for values crossing a JSON boundary.**

## 9.4 Idempotency rules

- Deduplicate every webhook on `webhookId` before any side effect.

- Upsert transactions on `GHl _id` with `ON CONFLICT DO UPDATE`.
- Apply an update only when the incoming `updatedAt` is newer than the stored value, so a late event cannot overwrite fresher state.
- Use `/contacts/upsert` and `/opportunities/upsert` rather than `create-then-handle-duplicate`.
- Acknowledge webhooks quickly with HTTP 200 and process asynchronously; slow handlers cause redelivery.
- Filter on `LiveMode` so test traffic never reaches production records.

## 10. Platform Constraints Affecting the Intended Design

The following are properties of the platform, established from the specifications above. They are recorded here because each one materially affects the architecture and each is cheaper to design around now than to discover during build.

### 10.1 Client-side address masking is presentation, not access control

If a public page renders property cards by calling the GHL API from the visitor's browser and hiding the exact address in the DOM, the address is still delivered to the browser. Anyone can read it from the network tab. The same applies to any financial field intended to be revealed only after the fee agreement is signed. Hiding a value in the client does not gate it.

There is a second, more serious form of this problem. GHL API credentials are bearer tokens scoped to an entire sub-account. A credential embedded in public JavaScript to fetch listings is readable by every visitor and grants that visitor the token's full scope — contacts, invoices, transactions — not merely the listing fields the page displays. Public browser code must never hold a GHL token.

The sound pattern:

- A server-side component holds the GHL credential and is the only thing that talks to GHL.
- It projects listings into the `property_listing` read model, splitting public fields from restricted ones at write time.
- The public page is served only the public projection. Restricted fields are never serialised into an unauthenticated response.
- Address reveal is a server-side authorisation decision, checked per request against the viewer's verified entitlement, not a CSS class or a JavaScript branch.

### 10.2 Portal gating is a marketing boundary, not a security boundary

Membership gating and tag-based visibility are appropriate for shaping what a logged-in user is shown. They are not designed as an authorisation boundary for confidential financial data belonging to a specific investor or agent. Where the requirement is that an agent can access only their assigned properties and conversations, that entitlement should be enforced server-side on every request that returns restricted data.

### 10.3 Public browsing cannot be served directly from the GHL API

As set out in section 5, the 100-request / 10-second burst limit is shared across all concurrent visitors because it is scoped per application per location. A marketplace whose search and filter operations hit GHL per visitor will degrade under modest traffic and cannot be fixed by tuning. Serving the front end from a cached read model, refreshed by webhooks, is the structural answer.

### 10.4 Transactional writes are constrained

Restated because it shapes the fee flow: no transaction can be created through the API. The platform fee must be represented as an invoice, an order payment record, or a charge through a registered custom payment provider. Any design that assumes arbitrary financial records can be pushed into GHL will need rework.

### 10.5 External listing status scraping

A nightly job that checks third-party listing portals for status changes sits outside GHL entirely and carries its own considerations: those sites' terms of use, active anti-automation measures, and the ongoing maintenance burden of

selectors that change without notice. Treat it as a distinct component with its own reliability expectations, and prefer a licensed data feed where one is available. Its output reaches GHL as an inbound webhook or a direct record update.

## 10.6 Summary

| Constraint                           | Consequence                     | Mitigation                                               |
|--------------------------------------|---------------------------------|----------------------------------------------------------|
| Bearer tokens are sub-account scoped | Cannot be exposed to browsers   | Server-side integration component holds all credentials  |
| Burst limit is per app, not per user | Public browsing will throttle   | Cached read model refreshed by webhooks                  |
| Client-side hiding is not gating     | Restricted data leaks           | Split public and restricted projections server-side      |
| Transactions are read-only           | Fees must be invoices or orders | Model the fee agreement as an invoice                    |
| Webhooks are not exactly-once        | Silent data drift               | Dedupe on webhookId plus nightly reconciliation          |
| Access tokens expire in ~24h         | Unattended jobs fail            | Private Integration Token, or atomic refresh persistence |

## 11. Error Handling

| Condition                      | Handling                                                                                      |
|--------------------------------|-----------------------------------------------------------------------------------------------|
| 401 Unauthorized               | Token expired or revoked. Refresh and retry once; if it recurs, alert. Never retry in a loop. |
| 403 Forbidden                  | Missing scope. Not retryable. Log the required scope explicitly.                              |
| 404 Not Found                  | Object deleted or wrong location scope. Reconcile rather than retry.                          |
| 422 Unprocessable              | Payload rejected. Log full request and response; requires a code fix.                         |
| 429 Too Many Requests          | Jittered exponential backoff; respect the rate-limit headers.                                 |
| 5xx                            | Retry with backoff and capped attempts, then dead-letter the job.                             |
| Missing Version header         | Deterministic failure. Enforce in the HTTP client, not per call site.                         |
| Signature verification failure | Return 401 and record the attempt. Never process an unverified payload.                       |

Carry a correlation id on every outbound call and log it alongside the GH L `traceId` where one is returned, so a support conversation with GH L can be grounded in a specific request.

## 12. Local Development and Testing

This phase is local-only. Nothing in the development environment is exposed to the internet, and nothing needs to be.

| Concern                | Local approach                                                                                       |
|------------------------|------------------------------------------------------------------------------------------------------|
| Database               | PostgreSQL 16 on <code>localhost:5432</code> , bound to <code>localhost</code>                       |
| Credentials            | Private Integration Token in an environment variable; never committed                                |
| Outbound calls to GH L | Work normally; only inbound delivery is restricted                                                   |
| Webhook receipt        | Cannot be delivered to this host. Replay recorded fixtures at the handler                            |
| Signature testing      | Unit-test <code>verify()</code> with a locally generated RSA keypair, plus one captured real payload |
| Contract testing       | Validate request builders against the vendored OpenAPI specifications                                |
| Live sandbox           | Use a GH L test sub-account and <code>paymentMode=test</code>                                        |
| Migration rehearsal    | Run the full pass 0–7 load against a disposable sub-account before production                        |

The only capability genuinely unavailable locally is receiving live webhook deliveries, which requires an inbound public URL. Build the receiver as a plain function over `(raw_body, headers)` so it can be driven from fixtures in tests and mounted on an HTTP route later without change. That keeps the deferred internet work a routing concern rather than a redesign.

**Treat the local database as disposable. Schema belongs in committed migrations and seed data in committed fixtures.**

## 13. Open Items and Decisions Required



| # | Item                                                                                | Next step                                       |
|---|-------------------------------------------------------------------------------------|-------------------------------------------------|
| 1 | Confirm the webhook signature algorithm and padding against a captured live payload | Engineering — blocks trusting the receiver      |
| 2 | Choose credential type: Private Integration Token or full OAuth app                 | Recommendation: Private Integration Token       |
| 3 | Decide where restricted property fields are authorised and served from              | Architecture — see section 10.1                 |
| 4 | Confirm which payment provider is connected in GHL                                  | Operations — constrains the fee flow            |
| 5 | Define the refund and partial-payment policy for the platform fee                   | Finance and engineering                         |
| 6 | Establish the GHL test sub-account and location id                                  | Operations                                      |
| 7 | Complete the EspoCRM field-level mapping to custom object schemas                   | Engineering — precedes pass 1                   |
| 8 | Decide the source of external listing status data                                   | Product — licensed feed preferred over scraping |

## 14. Sources and Verification

Endpoint paths, HTTP methods, OAuth scopes, the required version header, request and response field names, rate limits and the webhook event catalogue were read directly from GoHighLevel's official OpenAPI specification repository rather than from narrative documentation.

| Source                          | Reference                                                                                                                                     |
|---------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| Official OpenAPI specifications | <a href="https://github.com/GoHighLevel/highlevel-api-docs">github.com/GoHighLevel/highlevel-api-docs</a>                                     |
| Developer portal                | <a href="https://marketplace.gohighlevel.com/docs/">marketplace.gohighlevel.com/docs/</a>                                                     |
| Transactions API                | <a href="https://marketplace.gohighlevel.com/docs/ghl/payments/transactions/">marketplace.gohighlevel.com/docs/ghl/payments/transactions/</a> |
| OAuth 2.0 guide                 | <a href="https://marketplace.gohighlevel.com/docs/Authorization/OAuth2.0/">marketplace.gohighlevel.com/docs/Authorization/OAuth2.0/</a>       |
| Webhook authentication          | docs/oauth/webhookAuthentication.md in the spec repository                                                                                    |
| Rate limits                     | docs/oauth/Authorization.md in the spec repository                                                                                            |
| Product site                    | <a href="https://www.gohighlevel.com">www.gohighlevel.com</a>                                                                                 |

*Verification note. The rendered documentation host [marketplace.gohighlevel.com](https://marketplace.gohighlevel.com) was unreachable from the environment in which this document was prepared, so the specification repository was cloned from GitHub and read at source — the authoritative origin for the rendered documentation in any case. Two items are marked unverified rather than asserted: the webhook signature algorithm and padding (section 8.2), and the formal deprecation status of the legacy v1 API, which is absent from the current v2 specification but whose end-of-life date was not confirmed.*